

Midterm 1 Review Sheet

CS 5090 · Software Engineering with Generative AI · review session Monday, October 5 · exam Friday, October 9, in class

The exam at a glance

- **When:** Friday, October 9, during class (50 minutes). Arrive on time.
- **Rules:** closed book, closed notes, no phones, laptops, or AI tools. No calculator needed. A formula sheet is printed on the exam: softmax, attention, cross-entropy, top-p, the Chinchilla rule, pass@k, and values such as e^x and $\ln 2$. You do not need to memorize formulas; you need to know how to use them.
- **Format:** 15 multiple-choice questions, 10 true/false statements, three work-it-out problems (show your work for partial credit), three short-answer questions, and a small bonus section that is the hardest part of the exam.
- **Scope:** the lectures of Weeks 1–5, from "what is a language model" through RLHF, reasoning models and Codex, **and the in-class worksheets** for Weeks 3, 4 and 5. Not on the exam: the 🤖 AI-assisted SE sessions, and anything from Week 7 onward (retrieval, agents, evaluation).
- **Best preparation:** redo the three worksheets without looking at the answers, then check yourself against the posted answer sheets (Worksheets page on the course website). Most exam questions are close to a worksheet question or something worked in class.

What you should be able to do

Weeks 1–2 · Language models and the Transformer

- Define a language model (a probability distribution over token sequences) and trace the pipeline: text → tokenizer (BPE) → embeddings → N Transformer blocks → linear layer to $|V|$ → softmax → next token.
- Compare word, subword and character tokenization, and say why code models use subwords.
- Explain top-k and top-p sampling, and apply top-p to a small distribution.
- Describe the training loop (forward, loss, backward, update) and the sliding-window way training examples are cut from text.
- Tell the story from fixed-window networks to RNNs, seq2seq, attention and the Transformer: what each step fixed, and what the Transformer removed (recurrence) and gained (parallel training).
- Explain multi-head attention and what W_O is for.

Week 3 · Inside the block, decoding, prompting

- Masked (causal) self-attention vs cross-attention; draw a causal mask; compute attention weights from scores with a softmax.
- Residual connections, layer norm (and why not batch norm), the feed-forward network (and why it must be non-linear), position encodings (and why fixed sinusoids).
- Greedy vs beam search vs sampling; guided (constrained) decoding and why it matters for code.
- Prompting: zero-shot, few-shot, chain of thought, self-consistency, tree-of-thought; pros and cons; what happens when a prompt is unclear.
- What the KV cache stores and why it makes generation cheaper.
- Encoder-only (BERT), decoder-only (GPT), encoder–decoder: what each is used for.

Week 4 · Pretraining, scaling laws, fine-tuning

- Pretraining objectives (next token, masked token) and why no labels are needed; why pretraining won, and its downsides.
- Scaling laws: loss falls predictably with compute, parameters and data; the Chinchilla rule $D \approx 20N$; apply it to a model size.
- Predict the initial loss of an untrained model: $\approx \ln |V|$.
- SFT and instruction tuning: what changes (the data) and what does not (the loss). The ClarifyCoder case study: the problem, the synthetic data, the fine-tuning.
- LoRA (frozen base, trainable low-rank $\Delta W = BA$) and QLoRA; why they fit on small GPUs; overfitting signs in loss curves.

Week 5 · RLHF, reasoning models, Codex

- The pipeline pretraining → SFT → RLHF, and what each learns from. SFT imitates a correct answer; RL tries an answer and is told how good it is.

- InstructGPT's three steps: SFT on demonstrations; a reward model trained on human rankings of several responses (same LLM, but it outputs one scalar); PPO, which uses the reward model's score, not the preference pairs.
- PPO: increase reward while limiting how much the policy changes per step. DPO: optimize the preferences directly, with no reward model and no PPO.
- Reasoning: intermediate computation to reach an answer; fluent reasoning $\neq$ correct reasoning.
- Test-time scaling (think longer, e.g. budget forcing with "Wait" in s1; sampling; search; verification) vs train-time scaling (RLVR, GRPO, reasoning SFT).
- RLVR: reward 1 if the answer is verified correct, else 0. GRPO: what DeepSeek-R1-Zero used; it eliminates the critic model, which cuts the cost of RL.
- Codex and HumanEval: the training data, what a HumanEval problem contains, how correctness is judged, pass@k, and Codex's limitations.

Post-training at a glance

Stage	Data	Signal	What it changes
Pretraining	trillions of unlabeled tokens	next-token cross-entropy	knowledge, language, code
SFT / instruction tuning	(instruction, response) pairs	the same next-token loss	behaves like an assistant
RLHF (PPO)	human rankings $\rightarrow$ reward model	reward-model score; PPO limits each update	helpful, preferred style
DPO	the same comparisons	direct loss: chosen above rejected	same goal, no RM, no RL loop
RLVR (GRPO)	prompts with checkable answers	1 if verified correct, else 0	reasoning ability

Numbers and formulas worth knowing

- Attention: $\text{softmax}(QK^T / \sqrt{d_k}) V$. The softmax inside attention is over *positions*; the one at the output is over the *vocabulary*.
- Initial loss $\approx \ln |V|$ (e.g. $|V| = 8,192$ gives ≈ 9.0). Useful: $\ln |V| = (\text{number of bits}) \times \ln 2$ when $|V|$ is a power of 2.
- Chinchilla: tokens $\approx 20 \times$ parameters. Training compute $C \approx 6ND$.
- LoRA on a $d \times k$ matrix with rank r trains $r(d + k)$ parameters instead of dk .
- pass@k: a problem counts as solved if any of k samples passes the tests; the score is the fraction of problems solved.

Practice problems

Try these before Monday; we will go through several on the board. Answers are on the last page.

P1. A 3-token sequence. For the query at t2 the scaled scores against t1, t2, t3 are [0, 2, 4]. Which score is masked? Compute the attention weights of t2 ($e^2 \approx 7.39$).

P2. Next-token probabilities: a 0.60, b 0.20, c 0.10, d 0.06, e 0.04. Which tokens can be sampled with (i) top-k, $k = 2$? (ii) top-p, $p = 0.9$?

P3. A 25M-parameter model with a 32,768-token vocabulary ($= 2^{15}$). (a) Compute-optimal training tokens? (b) Expected first training loss? (c) With 2B tokens available, is 25M too big or too small?

P4. LoRA with rank $r = 8$ on one 768×768 weight matrix. How many parameters are trained, and what fraction of the matrix is that?

P5. A model generates 3 samples for each of 4 problems. Passing samples: P1 none; P2 samples 1 and 3; P3 sample 2 only; P4 sample 3 only. Compute pass@1 (sample 1 only) and pass@3. Under RLVR, what are P2's three rewards?

P6. Test-time or train-time scaling? (i) RLVR with GRPO; (ii) appending "Wait" so the model keeps thinking; (iii) SFT on reasoning traces; (iv) sampling 10 solutions and running the tests.

P7. Name the training stage: (i) 10,000 (instruction, response) pairs, next-token loss; (ii) (prompt, chosen, rejected) triples, no reward model; (iii) 8 samples per prompt, reward 1 if the unit tests pass; (iv) two trillion tokens of web text and code.

P8. Your code assistant must output SQL that always parses. Why is "set temperature to 0" not enough, and what would you use instead?

Explain it in two or three sentences

- Why did the Transformer replace RNNs, and what did it give up?
- Why fixed sinusoidal position encodings rather than learned ones?
- Self-attention vs cross-attention in an encoder-decoder: can they be swapped?
- Why does the FFN need a non-linearity? What does the residual connection buy?
- Why is generating token 500 not $500\times$ as expensive as token 1?
- Why do industries like scaling laws? When would you deliberately "overtrain" a small model?
- Why does a pretrained model with a fresh classification head score at chance before fine-tuning?
- Why fine-tune with LoRA rather than fully on a laptop GPU?
- Why does InstructGPT ask labelers to rank responses instead of writing them?
- What does DPO remove from the RLHF pipeline?
- Why did DeepSeek-R1-Zero use GRPO rather than PPO?
- What does a HumanEval problem contain, and what can go wrong with Codex-generated code?
- If ten samples agree, is the prompt well specified? Is the code correct?

Answers to the practice problems

P1. t_3 's score (4) is masked, since t_3 is in the future. $\text{softmax}([0, 2]) = 1 / 8.39$ and $7.39 / 8.39 = \mathbf{0.12, 0.88}$ for t_1, t_2 .

P2. (i) {a, b}. (ii) Cumulative 0.60, 0.80, 0.90, so {a, b, c}: the smallest set whose total reaches p . Top-k fixes the *number* of candidates; top-p fixes the *probability mass*, so its set shrinks when the model is confident.

P3. (a) $20 \times 25\text{M} = 500\text{M}$ tokens. (b) $\ln 32,768 = 15 \times 0.693 \approx 10.4$. (c) Too small for the data: 2B tokens would be compute-optimal for about 100M parameters. (Training the small model longer is still reasonable if inference cost matters more than training cost.)

P4. $r(d + k) = 8 \times (768 + 768) = 12,288$ trained parameters, vs $768 \times 768 = 589,824$ in the frozen matrix: about 2%.

P5. $\text{pass}@1 = 1/4 = \mathbf{25\%}$ (only P2's first sample passes); $\text{pass}@3 = 3/4 = \mathbf{75\%}$ (P2, P3, P4). Same model, more samples: that is test-time scaling. P2's rewards: **[1, 0, 1]**.

P6. (i) train-time; (ii) test-time (budget forcing); (iii) train-time; (iv) test-time.

P7. (i) SFT / instruction tuning; (ii) DPO; (iii) RLVR with GRPO; (iv) pretraining.

P8. Temperature 0 (greedy) makes the output deterministic, not valid: the most likely token can still break the syntax. Guided (constrained) decoding masks every token the SQL grammar does not allow at each step, so the output parses by construction.